Lecture 18: Procedure Calling Conventions

CMPS 221 – Computer Organization and Design

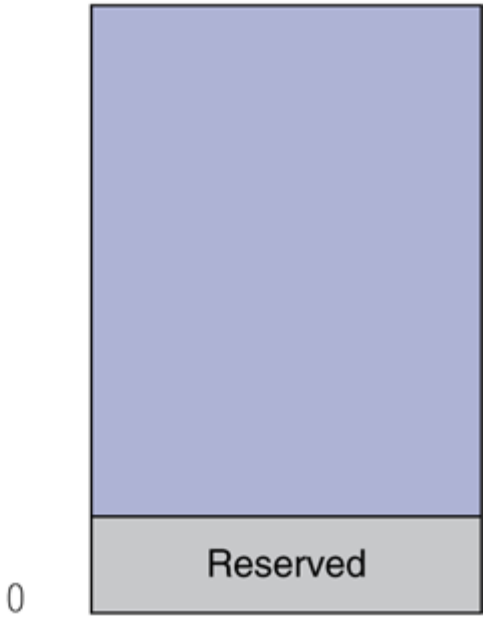
Adapted by Izzat El Hajj from textbook slides for Computer Organization and Design, by David Patterson and John Hennessy, Morgan Kaufmann.

Where to Back Up Data

- A procedure needs a place to:
  - Back up its **arguments**, **return address**, and **temporaries** if it is a caller
  - Back up its caller's **saved registers** if it needs to overwrite them
  - Place extra data that does not fit in the data registers (temporary and saved registers)
- Solution: use a stack in memory

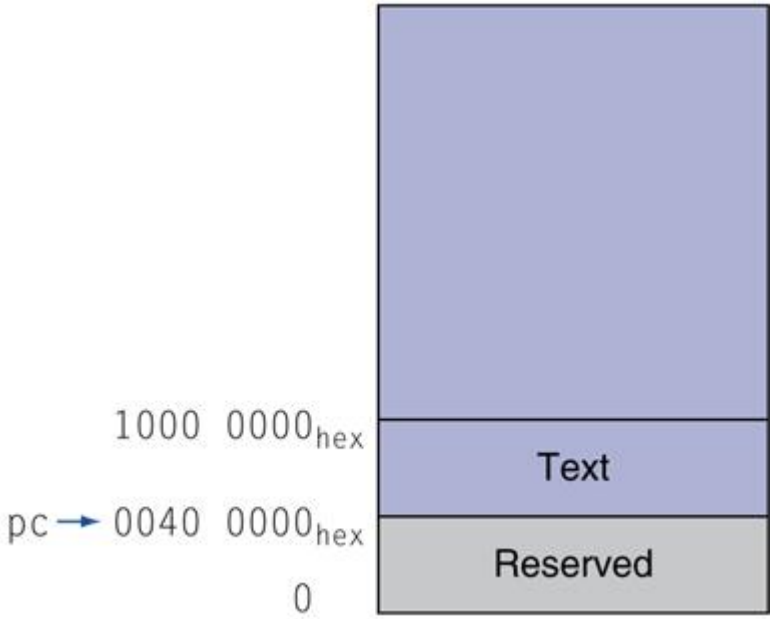
Memory Layout

- Reserved: for OS



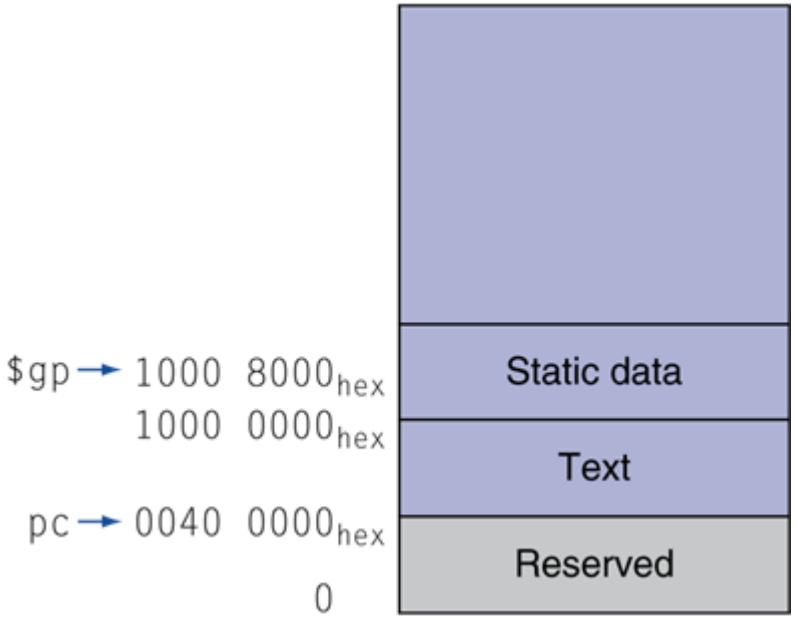
Memory Layout

- Reserved: for OS
- Text: program instructions



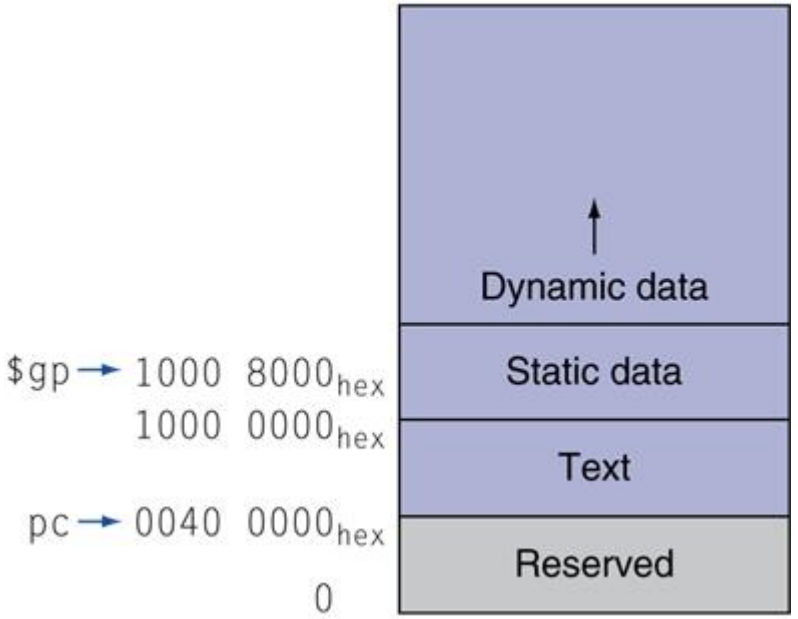
Memory Layout

- Reserved: for OS
- Text: program instructions
- Static data: global variables
  - e.g., constant arrays/strings, static variables in C
  - $gp initialized to address allowing ±offsets into this segment



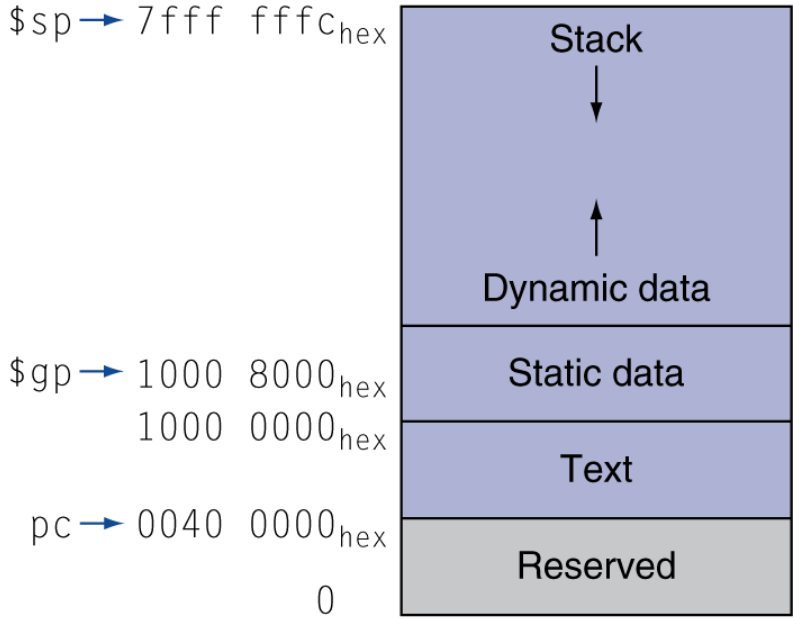
Memory Layout

- Reserved: for OS
- Text: program instructions
- Static data: global variables
  - e.g., constant arrays/strings, static variables in C
  - $gp initialized to address allowing ±offsets into this segment
- Dynamic data: heap
  - e.g., new in C++ and Java, malloc in C



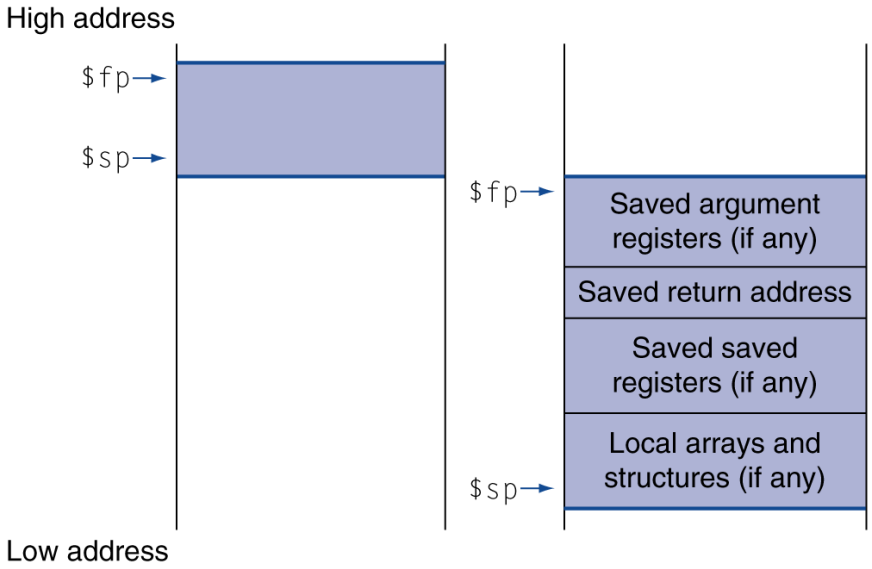
Memory Layout

- Reserved: for OS
- Text: program instructions
- Static data: global variables
  - e.g., constant arrays/strings, static variables in C
  - $gp initialized to address allowing ±offsets into this segment
- Dynamic data: heap
  - e.g., new in C++ and Java, malloc in C
- Stack: automatic storage
  - For procedure data



Local Data on the Stack

- Every procedure has a frame on the stack



  - $fp: frame pointer (reg 30)
    - Points to the beginning of a procedures stack frame
  - $sp: stack pointer (reg 29)
    - Points to the end of a procedures stack frame (top of the stack)
  - $fp is useful when $sp changes throughput a procedure
    - For simplicity, we will only use $sp and extend the stack only on procedure entry/exit

Procedure Calling Convention

- At function entry:
  - Allocate stack memory if needed
  - If function is a caller, back up $a_ and $ra
  - If function uses $s_ registers, backup its caller's $s_ registers
- At function exit:
  - Write return value to $v_ registers
  - Restore caller's $s_ registers
  - Deallocate stack memory
  - Return control to caller (jr $ra)

Procedure Calling Convention

- Before calling another function:
  - Write arguments to $a_ registers
  - Back up $t_ registers on the stack if needed
  - Transfer control to callee (jal)

- After calling another function:
  - Restore $a_, $ra, and $t_ registers
  - Obtain return value from $v_ registers

Leaf Procedure Example

- C code:

```
int leaf_example(int g, int h, int i, int j) {
    int f;
    f = (g + h) - (i + j);
    return f;
}
```

  - Arguments g, …, j in $a0, …, $a3
  - f in $s0
    - Need to save caller's $s0 on the stack
  - Result in $v0

Leaf Procedure Example

```
int leaf_example(int g, int h, int i, int j) {
    int f;
    f = (g + h) - (i + j);
    return f;
}
```

Label for caller to jump to	`leaf_example:`
Save $s0 on stack	`addi $sp, $sp, -4` `sw $s0, 0($sp)`

Procedure body	`add $t0, $a0, $a1`
	`add $t1, $a2, $a3`
	`sub $s0, $t0, $t1`

Result	`add $v0, $s0, $zero`
Restore $s0	`lw $s0, 0($sp)` `addi $sp, $sp, 4`
Return	`jr $ra`

Non-Leaf Procedure Example

- C code:

```
int fact (int n) {
    if (n < 1)
        return 1;
    else
        return n * fact(n - 1);
}
```

  - Argument n in $a0
    - Need to save $a0 and $ra on the stack so as not to overwrite them when we make a call
  - Result in $v0

Non-Leaf Procedure Example

Label	`fact:`

Save $a0, $ra	`addi $sp, $sp, -8` `# adjust stack for 2 items`
	`sw $ra, 4($sp)` `# save return address`
	`sw $a0, 0($sp)` `# save argument`

if(n<1) return 1	`slti $t0, $a0, 1` `# test for n < 1`
	`beq $t0, $zero, L1`
	`addi $v0, $zero, 1` `# if true, result is 1`

Return	`addi $sp, $sp, 8` `# pop 2 items from stack` `jr $ra` `# and return`
else …	`L1: addi $a0, $a0, -1` `# else decrement n`
Call fact	`jal fact` `# recursive call`
Restore $a0, $ra	`lw $a0, 0($sp)` `# restore original n` `lw $ra, 4($sp)` `# and return address`
n*fact(n-1)	`mul $v0, $a0, $v0` `# multiply to get result`
Return	`addi $sp, $sp, 8` `# pop 2 items from stack` `jr $ra` `# and return`

Textbook Sections

- The content in these slides corresponds to:
  - Textbook:
    - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
  - Sections:
    - 2.8